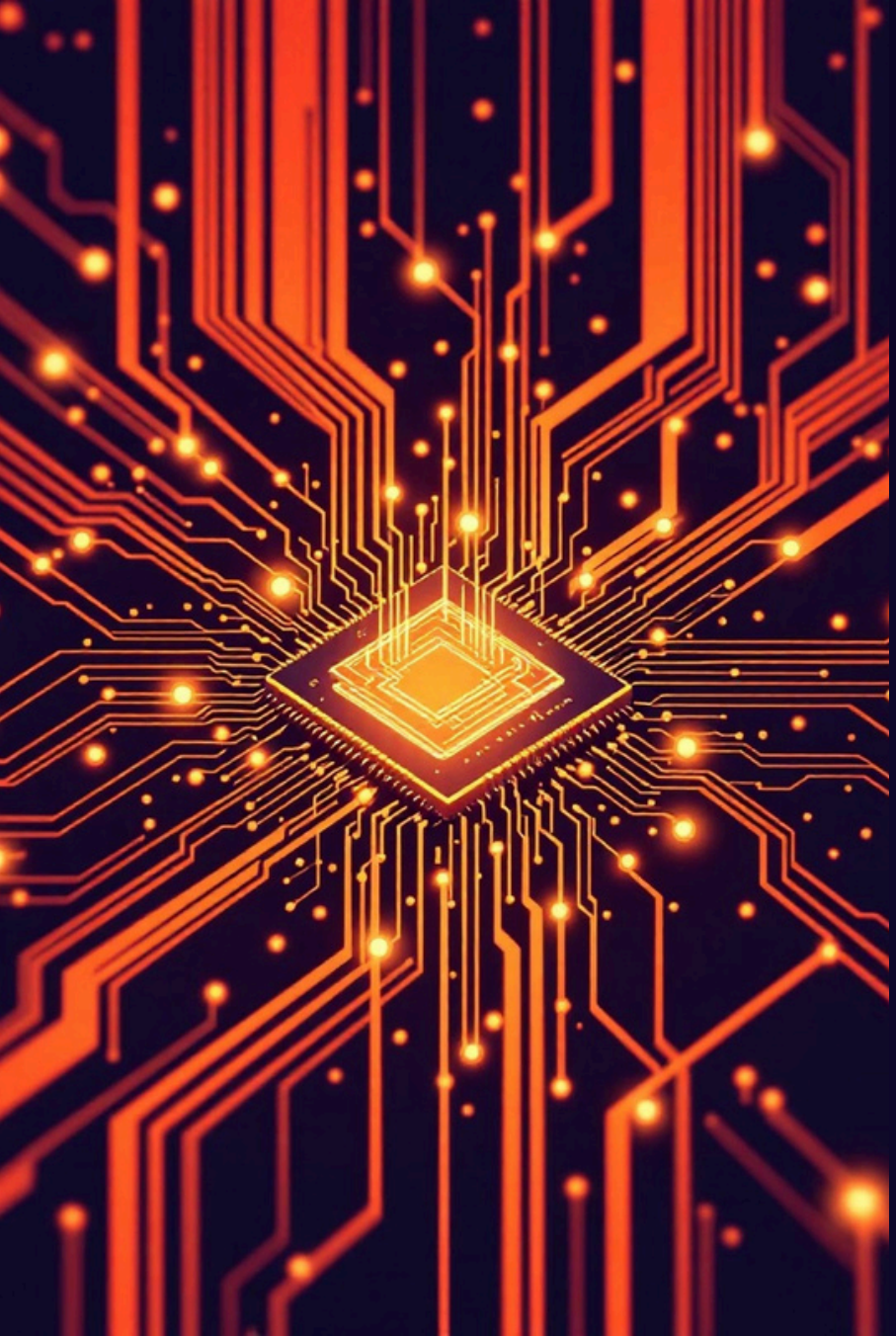


PROYECTO INTEGRADOR - COMPLEJIDAD COMPUTACIONAL

Importando Recursos Necesarios

Paranuestroanálisisdecomplejidadcomputacional,hemos importado las siguientes librerías esenciales:

Numpy

Para operaciones numéricas eficientes, especialmente con matrices.

Matplotlib

Para la visualización de datos y la creación de gráficos.

Time

Para medir el tiempo de ejecución de los algoritmos.

Random

Para la generación de números aleatorios en pruebas.



Problema 1: SLOWSORT vs QUICKSORT

Slowsort

Unalgoritmo de ordenación recursivo con una complejidad de $O(n \log n)$. Es extremadamente lento y no polinómico, demostrando un crecimiento factorial.

```
def slowsort(arr, i=0, j=None):
if j is None: j = len(arr) - 1
    if i >= j: return arr
 m = (i + j) // 2
arr = slowsort(arr, i, m)
 arr = slowsort(arr, m + 1, j)
 if arr[j] < arr[m]:
       arr[j], arr[m] = arr[m], arr[j]
 arr = slowsort(arr, i, j - 1)
   return arr
```

Quicksort

Utiliza la estrategia "divide y vencerás". Su complejidad promedio es $O(n \log n)$, lo que lo hace muy eficiente. El peor caso es $O(n2)$.

```
def quicksort(arr):
if len(arr) <= 1: return arr
pivot = arr[-1]
menores = [x for x in arr[:-1] if x < pivot]
iguales = [x for x in arr if x == pivot]
 mayores = [x for x in arr[:-1] if x > pivot]
 return quicksort(menores) + iguales + quicksort(mayores)
```

Problema 2: DETERMINANTE vs PERMANENTE

Determinante Recursivo

Calcula el determinanteusandola expansión por cofactores (Laplace). Su complejidad es $O(n!)$, lo que lo hace inviable para matrices grandes.

```
def determinante_recursivo(matriz):
n = len(matriz)
if n == 1: return matriz[0][0]
 if n == 2: return matriz[0][0] * matriz[1][1] - matriz[0][1] *
matriz[1][0]
   det = 0
   for j in range(n):
      submatriz = [fila[:j] + fila[j+1:] for fila in matriz[1:]]
      cofactor = ((-1) ** j) * matriz[0][j] *
determinante_recursivo(submatriz)
      det += cofactor
   return det
```

Permanente

Similaraldeterminante, pero con una complejidad de $O(n!çn)$. No existen atajos conocidos, lo que lo clasifica como #P-completo.

```
def permanente(matriz):
n = len(matriz)
if n == 1: return matriz[0][0]
 perm = 0
 for j in range(n):
 submatriz = [fila[:j] + fila[j+1:] for fila in matriz[1:]]
 perm += matriz[0][j] * permanente(submatriz)
return perm
```

Problema 3: TEST DE PRIMALIDAD SOLOVAY-STRASSEN

Símbolo de Jacobi

Función auxiliar parael test, calcula el símbolo de Jacobi (a/n). Es un paso crucial en la verificación de la primalidad.

```
def simbolo_jacobi(a, n):
if n <= 0 or n % 2 == 0: return 0
a = a % n
   resultado = 1
 while a != 0:
      while a % 2 == 0:
         a //= 2
         if n % 8 in [3, 5]: resultado = -resultado
      a, n = n, a
      if a % 4 == 3 and n % 4 == 3: resultado = -resultado
      a = a % n
   return resultado if n == 1 else 0
```

Solovay-Strassen

Untestde primalidadprobabilístico con complejidad $O(k$çlog3$n)$. Es muy eficiente para probar números grandes, clasificándose en la clase RP.

```
def solovay_strassen(n, k=20):
if n <= 1: return False
if n <= 3: return True
if n % 2 == 0: return False
for _ in range(k):
 a = random.randint(2, n - 1)
 x = simbolo_jacobi(a, n)
 if x == 0: return False
 y = pow(a, (n - 1) // 2, n)
if x % n != y: return False
return True
```

Problema 4: TORRES DE HANOI

Un clásico ejemplo de recursión con complejidad $O(2^n)$. El número de movimientos crece exponencialmente con el número de discos.

Función `torres_hanoi`

Mueve $n-1$ discos de origen a auxiliar, luego el disco más grande de origen a destino, y finalmente los $n-1$ discos de auxiliar a destino.

```
def torres_hanoi(n, origen='A', destino='C', auxiliar='B', movimientos=None):
if movimientos is None: movimientos = []
    if n == 1:
        movimientos.append((origen, destino))
        return movimientos
 torres_hanoi(n - 1, origen, auxiliar, destino, movimientos)
 movimientos.append((origen, destino)) torres_hanoi(n - 1, auxiliar, destino, origen, movimientos) return movimientos
```

Configuración Final

Retorna la configuración final de las torres y la lista de movimientos realizados.

```
def configuracion_hanoi(n):
movimientos = torres_hanoi(n)
torres = {
'A': list(range(n, 0, -1)),
'B': [],
 'C': []
 }
 for origen, destino in movimientos:
 disco = torres[origen].pop()
torres[destino].append(disco)
return torres, movimientos
```

Problema 5: ALGORITMO DE FLEURY (Camino Euleriano)

Este algoritmo encuentra un camino Euleriano en un grafo, si existe. Su complejidad es $O(E2)$, donde E es el número de aristas.

Función `es_puente`

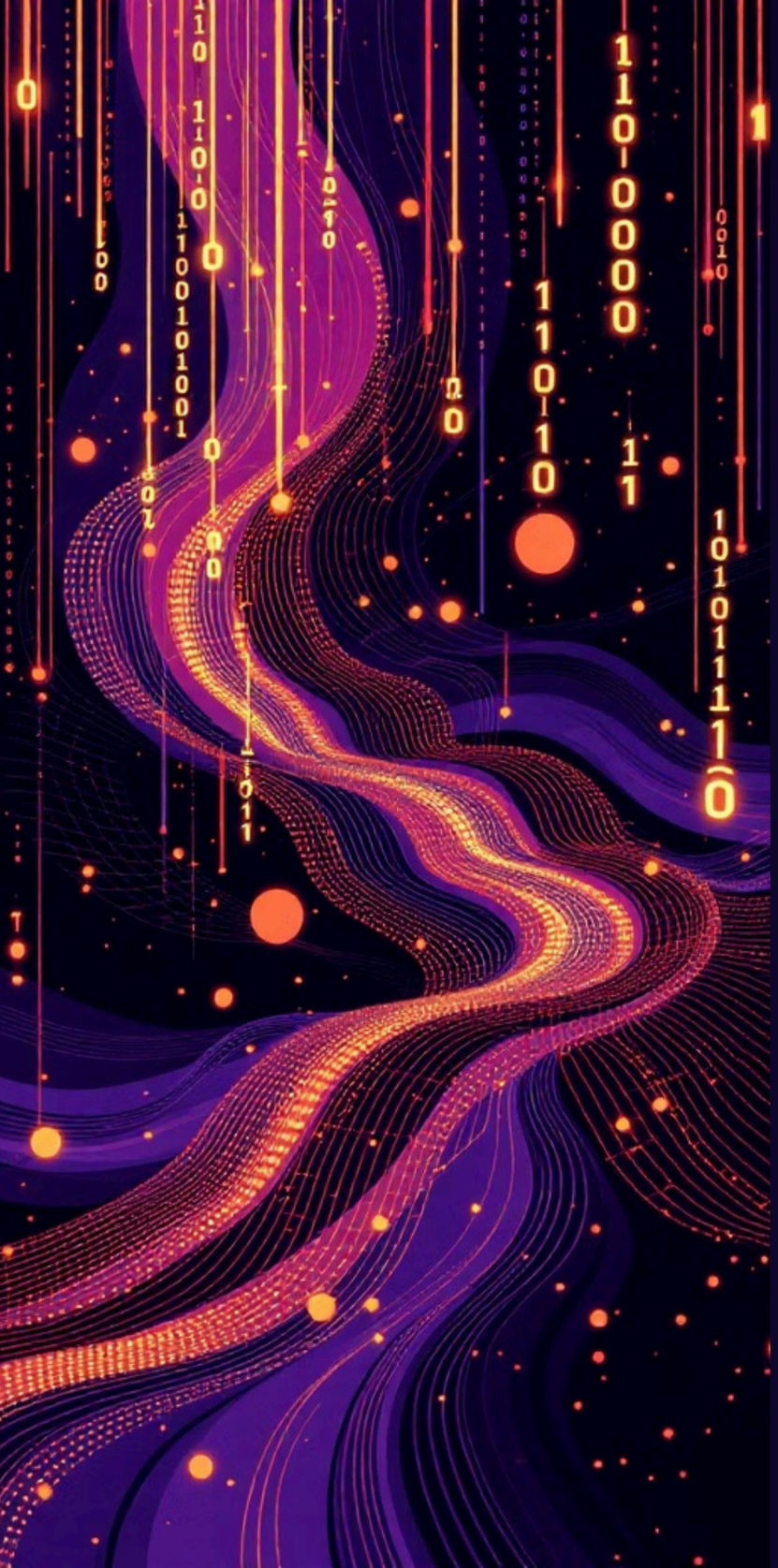
Determina si una arista es un puente en el grafo, lo cual es crucial para el algoritmo de Fleury.

```
def es_puente(grafo, u, v):
    grafo[u].remove(v)
    grafo[v].remove(u)
    visitados = set()
    def dfs(nodo):
        visitados.add(nodo)
        for vecino in grafo[nodo]:
            if vecino not in visitados: dfs(vecino)
    nodos = [n for n in grafo if grafo[n] or any(n in grafo[x] for x in grafo)]
    if nodos: dfs(nodos[0]) es_puente_resultado = len(visitados) < len(nodos) grafo[u].append(v)
    grafo[v].append(u) return es_puente_resultado
```

Algoritmo de Fleury

Comienza en un nodo con grado impar (o cualquiera) y elige aristas que no sean puentes, si es posible.

```
def fleury(grafo_original): grafo = {k: v[:] for k, v in grafo_original.items()} grados_impares = [v for v in grafo if len(grafo[v]) % 2 == 1] if len(grados_impares) not in [0, 2]: return None inicio = grados_impares[0] if grados_impares else list(grafo.keys())[0] camino = [inicio] actual = inicio while any(grafo[v] for v in grafo): for vecino in grafo[actual][:]: if len(grafo[actual]) == 1 or not es_puente(grafo, actual, vecino): grafo[actual].remove(vecino) grafo[vecino].remove(actual) camino.append(vecino) actual = vecino break return camino
```



Problema 6: CIFRADO XOR

Elcifrado XOR esunmétodo simple yeficientepara cifrarydescifrar mensajes. Su complejidades $O(n)$, linealrespectoa lalongituddelmensaje.

Cifrado XOR

Aplica la operación lógica XOR a nivel de bits entre el mensaje y una clave. Es muy eficiente debido a su naturaleza lineal.

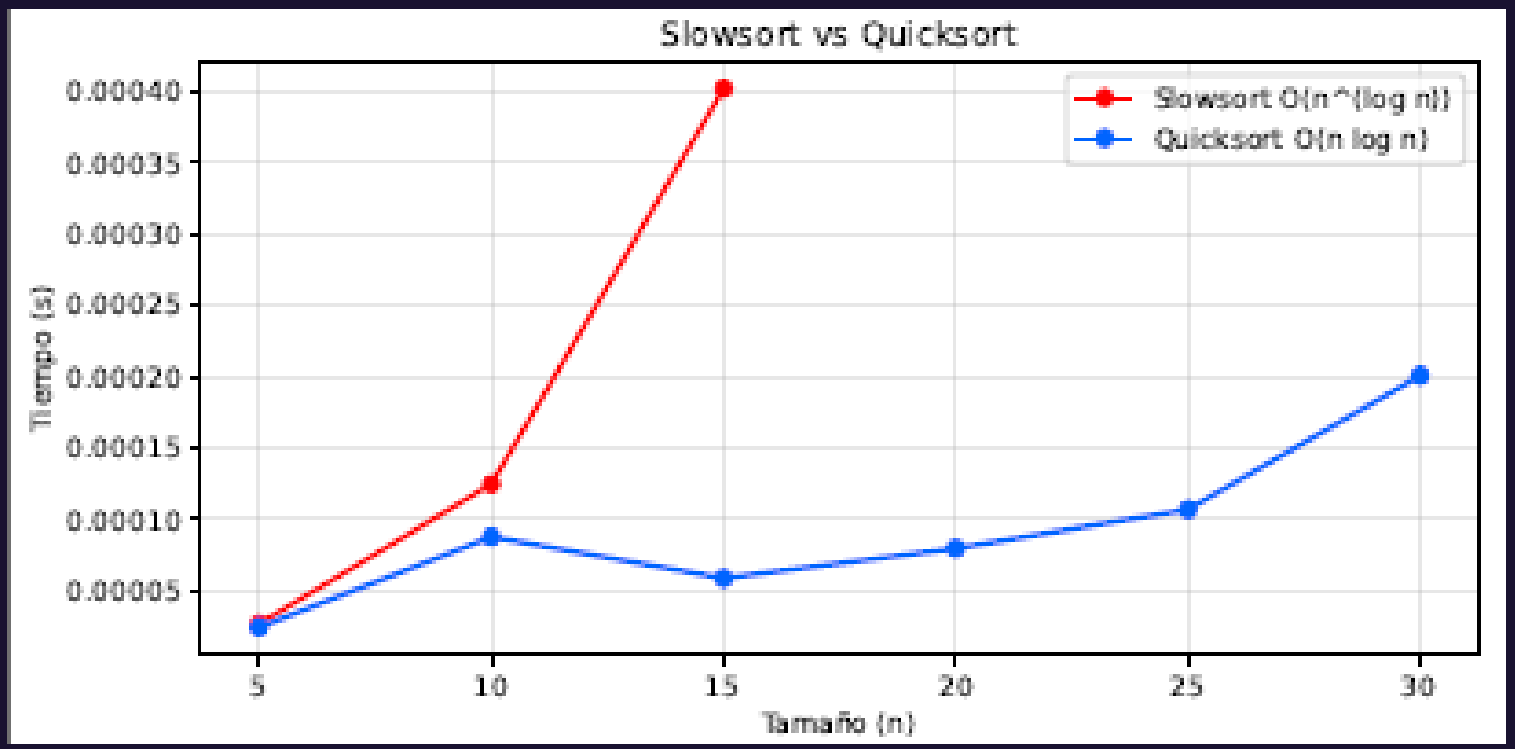
```
def xor_cifrado(mensaje, clave):
    resultado = []
  for i, char in enumerate(mensaje): clave_char =
clave[i % len(clave)] resultado.append(chr(ord(char) ^
ord(clave_char)))
    return ''.join(resultado)
```

Descifrado XOR

El proceso de descifrado es idéntico al de cifrado, ya que la operación XOR es su propia inversa.

```
def xor_descifrado(mensaje_cifrado, clave):
return xor_cifrado(mensaje_cifrado, clave)
```

ANÁLISIS Y GRAFICACIÓN

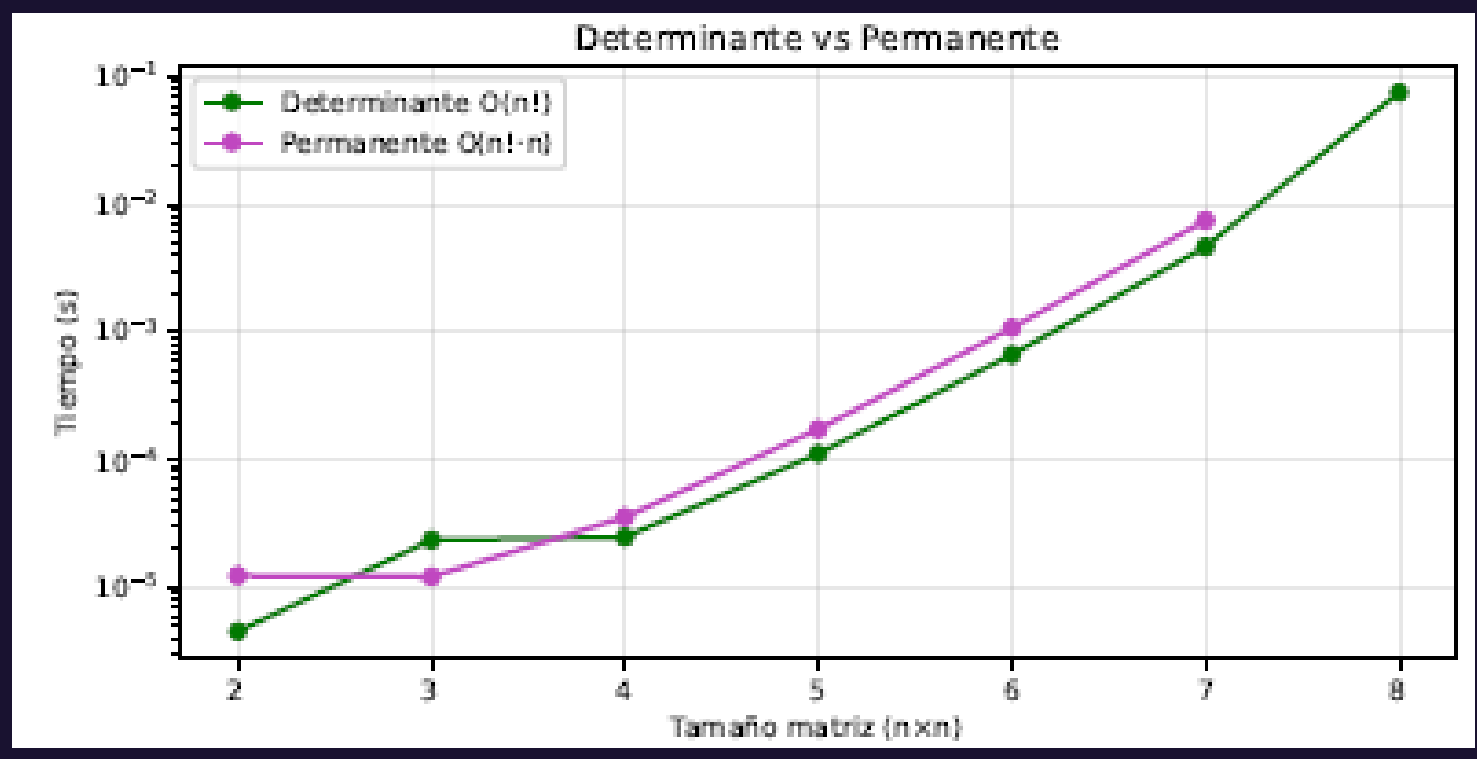


Slowsort vs Quicksort

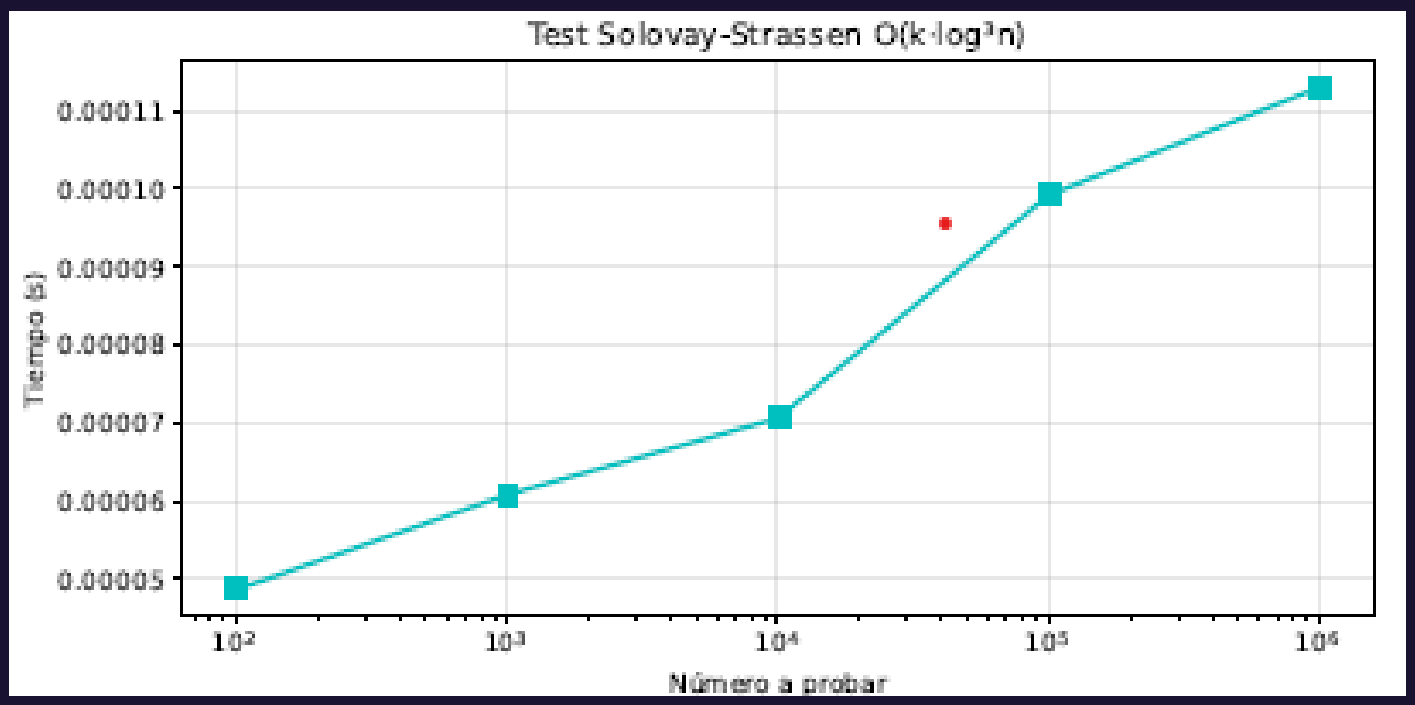
Comparación de algoritmos de ordenación, destacando la eficiencia de Quicksort.

Determinante vs Permanente

Análisis de funciones matriciales, revelando la alta complejidad del Permanente.



ANÁLISIS Y GRAFICACIÓN

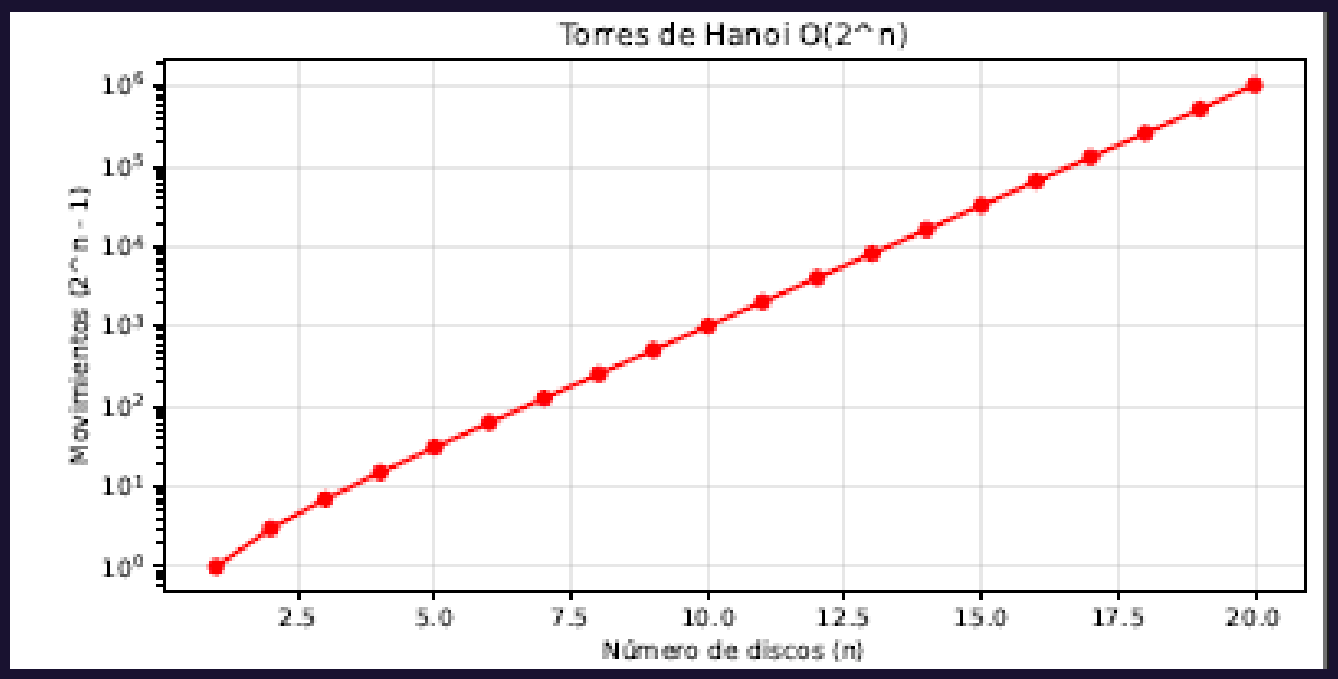


Solovay-Strassen

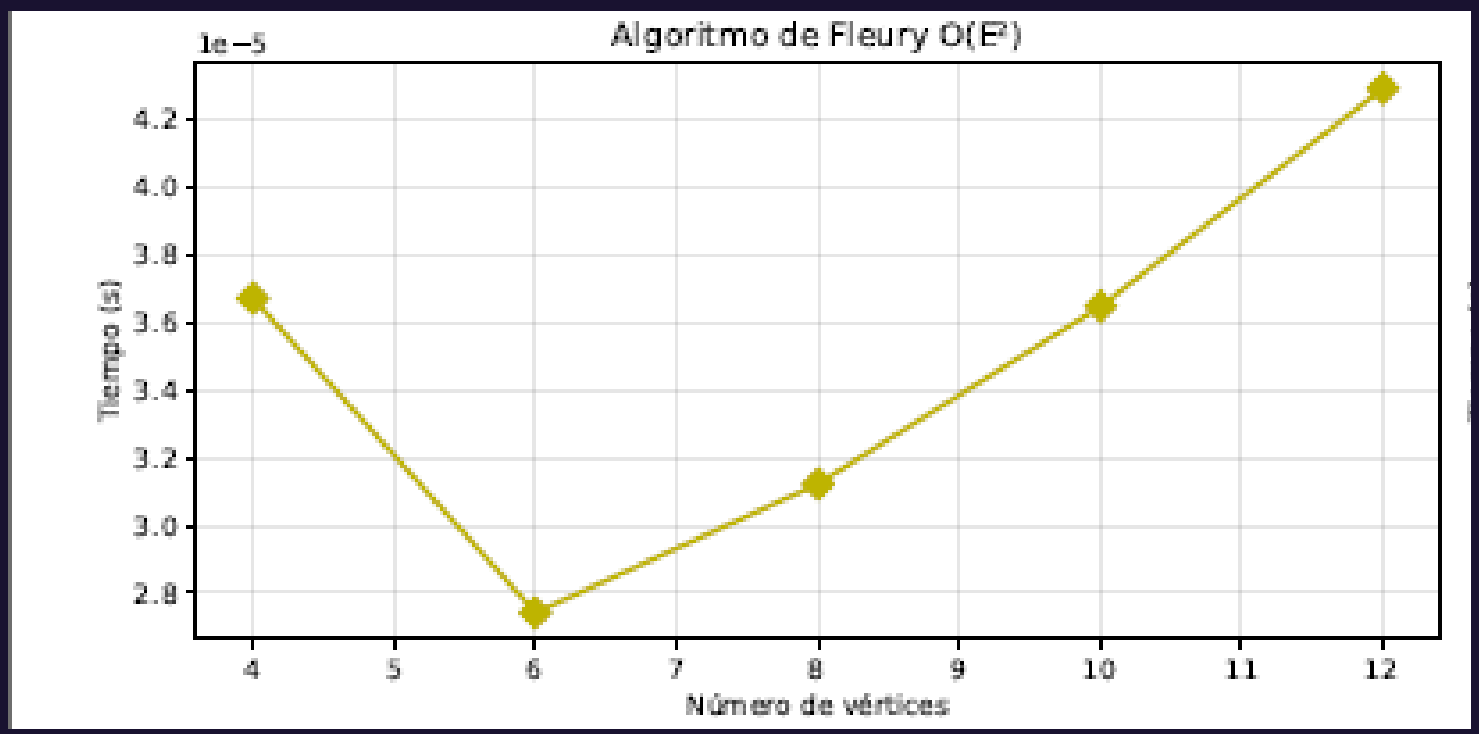
Rendimiento del test de primalidad, mostrando su eficiencia logarítmica.

Torres de Hanoi

Visualización del crecimiento exponencial de movimientos con el número de discos.



ANÁLISIS Y GRAFICACIÓN

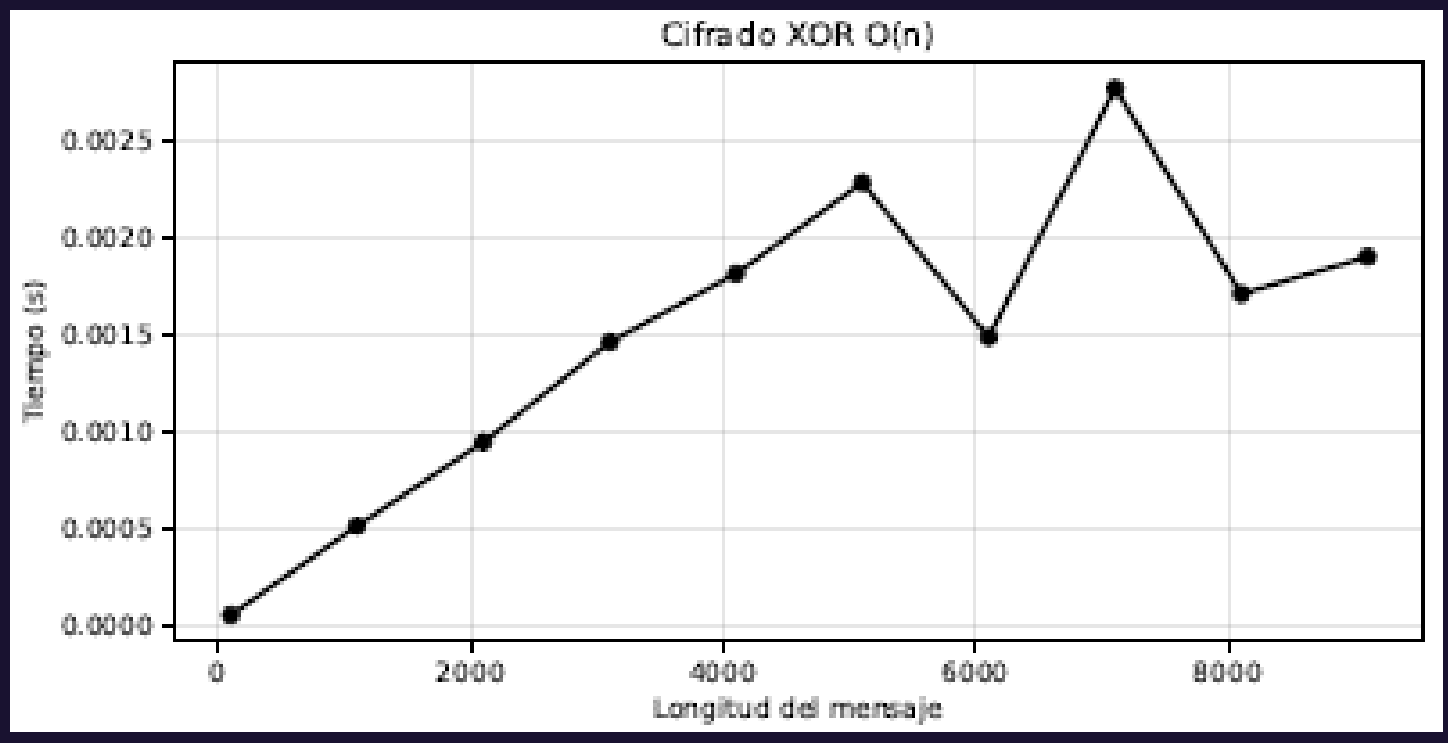


Algoritmo de Fleury

Análisis de la complejidad polinómica del algoritmo de caminos Eulerianos.

Cifrado XOR

Demostración de la eficiencia lineal del cifrado y descifrado XOR.



Resultados Clave del Análisis

1 Slowsort vs Quicksort

Quicksort ordena eficientemente `[5,2,8,1, 9]` a `[1, 2, 5, 8, 9]`, mientras que Slowsort, aunque correcto, es mucho más lento.

2 Determinante vs Permanente

Para una matriz `[[1, 2], [3, 4]]`, el Determinante es `-2` y el Permanente es `10`, ilustrando sus diferentes complejidades.

3 Test Solovay-Strassen

Identifica correctamente `17, 97, 1009` como probablemente primos y `18, 100` como compuestos.

4 Torres de Hanoi

Con `3` discos, se requieren `7` movimientos para alcanzar la configuración final `{'A': [], 'B': [], 'C': [3, 2, 1]}`.

5 Algoritmo de Fleury

Para el grafo `{0: [1, 2], 1: [0, 2], 2: [0, 1]}`, se encuentra el camino Euleriano `[0, 1, 2, 0]`.

6 Cifrado XOR

El mensaje `"Hola Mundo"` cifrado con la clave `"secreto"` se descifra correctamente, demostrando la reversibilidad del algoritmo.

Conclusión:

Este proyecto hademostrado que la verdadera **optimización de resultados** no comienza con la escritura de código más rápido, sino con la elección delalgoritmo correcto mediante el análisis de **complejidad computacional**.

A través de nuestras pruebas, hemos evidenciado que:

- **La optimización es estructural:** Ninguna cantidad de micro-optimizaciones en el código de *Slowsort* o el *Permanente* podría igualar el rendimiento nativo de *Quicksort* o el *Determinante*. La diferencia entre $O(n!)$ y $O(n \log n)$ es la diferencia entre un problema intratable y uno resuelto en milisegundos.
- **Escalabilidad es igual a éxito:** Los algoritmos eficientes como el **Cifrado XOR** $O(n)$ o el test de **Solovay-Strassen** $O(k \log 3n)$ garantizan que nuestros resultados se mantengan óptimos incluso cuando el volumen de datos crece exponencialmente.